HashiCorp Certified: Terraform Associate（004）　目標 1a・1b・1c　#01

IaCとTerraformのしくみ

IaC と宣言型、Write・Plan・Apply、プロバイダーと API、ステート、マルチクラウド・ハイブリッド・サービス非依存、Community Edition・HCP Terraform・Enterprise

各ページの下に、根拠にした資料（HashiCorp の Exam Content List（004）、Terraform ドキュメント v1.12.x の What is Terraform?・Terraform Use Cases・Core Terraform Workflow Overview・Terraform Editions Overview・Terraform versus Alternatives）を書いた。英語の原文は『』で引用した

全体像 コードで書いて、API で作る




```
resource "aws_vpc" "main" {
  cidr_block = var.base_cidr_block
}

<BLOCK TYPE> "<LABEL>" "<LABEL>" {
  <IDENTIFIER> = <EXPRESSION>  # 引数
}
```

> 『Terraform creates and manages resources on cloud platforms and other services through their application programming interfaces (APIs).』

出典：What is Terraform?／Terraform Language Documentation

第1章

IaC とその利点

手作業から、コードへ。

IaC と宣言型

方法	書くもの	弱み・強み
手作業・手作りスクリプト	手順	大きくなるとまちがいやすい。ツールの保守が足かせ
宣言型の IaC	最後のあるべき姿	手順はツールが考える。くり返し同じものを作れる
構成管理ツール	マシンの中のソフト	すでにあるマシンが対象。Terraform とは役割がちがう

> 『Terraform configuration files are declarative, meaning that they describe the end state of your infrastructure.』

バージョン管理
履歴・レビュー・元に戻す

再利用・共有
モジュールで組織の標準をそろえる

不変のインフラ
直すのではなく、作って置き換える

> 『Terraform is not a configuration management tool』（Chef・Puppet とは組み合わせて使う）

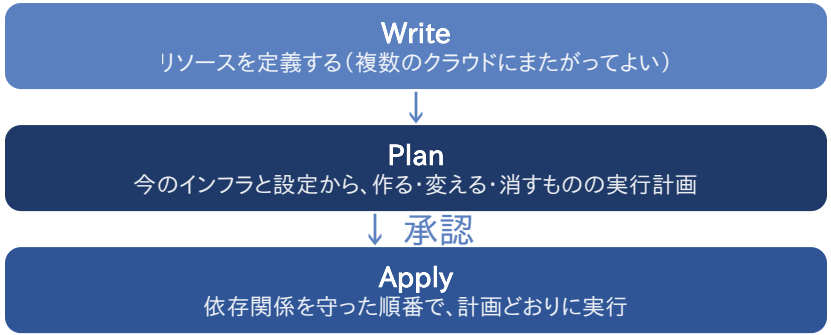
出典：What is Terraform?／Terraform vs. Chef, Puppet, etc.／Terraform versus Custom Solutions

第2章

Terraform のしくみ

Write・Plan・Apply とステート。

基本のワークフロー




```
# 一人で使うとき
$ git init my-infra && cd my-infra
$ vim main.tf
$ terraform init
$ terraform plan   # 書きながらくり返す
$ terraform apply
```

> 計画と実行を分けるので、『they know exactly what will happen beforehand』。

> 依存しないリソースは、リソースグラフで並列に作る。

出典：Core Terraform Workflow Overview／What is Terraform?／Terraform versus CloudFormation

構成要素

要素	役割
Terraform 言語（HCL）	ブロック・引数・式で書く。主な目的はリソースの宣言
プロバイダー	API のあるプラットフォーム・サービスを扱うプラグイン
Terraform Registry	公開されたプロバイダーとモジュールを探す（registry.terraform.io）
ステートファイル	実際のインフラの記録。『a source of truth for your environment』
リソースグラフ	依存関係から順番を決め、依存しないものは並列に
モジュール	設定を部品にして再利用。組織の標準をセルフサービスに

出典：What is Terraform?／Terraform Language Documentation／Terraform Use Cases

第3章

マルチクラウドとエディション

同じワークフローで、どこにでも。

いろいろな環境を同じやり方で

言葉	意味
マルチクラウド	複数のクラウド（AWS＋Azure など）
ハイブリッドクラウド	オンプレミス＋パブリッククラウド
サービス非依存	DNS・CDN・監視・SaaS も同じワークフロー
並行環境	開発・テスト・本番を作っては消す

> 『Terraform lets you use the same workflow to manage multiple providers and handle cross-cloud dependencies.』

CloudFormation
AWS の中だけ

Terraform
『both cloud-agnostic and enabling multiple providers and services to be combined and composed』

> 例：AWS と OpenStack のクラスタ、Cloudflare の CDN、DNSimple の DNS を 1 つの設定で。

出典：Terraform Use Cases／Terraform versus CloudFormation, Heat／What is Terraform?

3 つのエディション

エディション	形	特徴
Terraform Community Edition	無料・ダウンロードして CLI で使う	設定・プラグイン・インフラ・ステートを管理。最新の設定は自分で pull
HCP Terraform	HashiCorp の SaaS	リモート実行・ステートとシークレットの保管・RBAC・プライベート・レジストリ・VCS 連携・Sentinel
Terraform Enterprise	セルフホストの HCP Terraform	リソースの上限を変えられる。厳しいセキュリティ・コンプライアンスの要件向け

> Sentinel：『a policy-as-code framework, to automatically enforce compliance and governance policies before Terraform makes infrastructure changes』（HCP Terraform と Enterprise）

出典：Terraform Editions Overview／Terraform Use Cases（Policy Compliance and Management）

本番での問われ方

ねらい	見るところ
IaC とは	人が読める設定ファイル、バージョン管理・再利用・共有
宣言型	最後の姿を書く、手順と順番は Terraform が決める
ほかのツールとの比較	構成管理ツールではない、CloudFormation とちがいクラウドに依存しない
しくみ	API とプロバイダー、Write → Plan → Apply、ステートは source of truth
マルチクラウド	同じワークフロー、クラウドをまたぐ依存関係、ハイブリッドとのちがい
エディション	CLI・SaaS・セルフホストのちがい、Sentinel はどこで使えるか

出典：Exam Content List（004）1a・1b・1c／本資料の問題演習（全 30 問）